Guía de Uso de GitHub Enterprise

Empresa: TechRetail
Versión: 1.0
Fecha: Septiembre 2025

---

1. Objetivo

Establecer lineamientos para el uso seguro, eficiente y colaborativo de **GitHub Enterprise** en TechRetail, garantizando la gestión ordenada del código fuente, la trazabilidad de cambios y la integración con los procesos de desarrollo.

---

2. Alcance

Aplica a:

- Todos los desarrolladores, arquitectos y líderes técnicos de TechRetail.
- Repositorios de proyectos internos, servicios core, librerías compartidas y documentación técnica.

---

3. Acceso a GitHub Enterprise

- **URL corporativa:** https://github.techretail.com
- **Autenticación:** Single Sign-On (SSO) con credenciales corporativas + MFA.
- **Permisos:** gestionados por el área de Infraestructura TI.
- **Roles comunes:**
  - *Owner:* administra la organización y repositorios.
  - *Maintainer:* gestiona repositorios y equipos.
  - *Developer:* crea ramas, PRs y contribuye código.
  - *Viewer:* acceso de solo lectura.

---

4. Creación y gestión de repositorios

- Los repositorios deben crearse bajo la organización **TechRetail**.
- Nombres de repositorios en formato:

- `area-proyecto-modulo`

  Ejemplo: `finanzas-facturacion-api`

- Configuración inicial:
  - Archivo `README.md` con propósito y responsables.
  - Archivo `LICENSE` corporativo.
  - `.gitignore` estándar según tecnología.
  - Rama principal: **main**.

---

5. Flujo de trabajo estándar (Git Flow simplificado)

1. **Clonación del repositorio:**
```
git clone https://github.techretail.com/area/proyecto.git
```
3. **Creación de rama de feature:**
```
git checkout -b feature/nombre-funcionalidad
```
5. **Commits claros y frecuentes:**
   - Formato: `[Módulo] Descripción breve`
   - Ejemplo: `[Login] Fix validación de tokens expirados.`
6. **Pull Request (PR):**
   - Revisado por al menos **1 revisor** (Code Review).
   - Validación de pruebas unitarias y pipeline CI/CD.
7. **Merge a main:**
   - Solo después de aprobación y tests exitosos.
   - Se recomienda *Squash and Merge* para mantener historial limpio.

---

6. Seguridad y cumplimiento

- **Branch protection:** activada en `main` y `release/*`.
- **Revisiones obligatorias:** mínimo un aprobador.
- **Secrets Management:** usar GitHub Actions Secrets o Azure Key Vault (no subir claves al repositorio).
- **Dependabot:** habilitado para monitoreo de vulnerabilidades en dependencias.

---

7. Integración con pipelines y herramientas

- **CI/CD:** GitHub Actions integrado con Azure DevOps.
- **Automatización:** pruebas unitarias, análisis estático de código, despliegues automáticos.

- **Integraciones comunes:** Jira (issues), Slack/Teams (notificaciones), SonarQube (análisis de calidad).

---

8. Buenas prácticas

- Mantener repositorios privados por defecto.
- Usar Issues y Projects para gestión de tareas.
- Documentar arquitectura y decisiones en `docs/`.
- Revisar métricas de calidad periódicamente (SonarQube, CodeQL).
- Eliminar ramas obsoletas después de merge.

---

9. Procedimiento ante incidentes

- Si se detecta **exposición de credenciales** → revocar inmediatamente y notificar al área de Seguridad TI.
- Si se compromete un repositorio → congelar acceso, generar backup y activar plan de respuesta a incidentes.
- Reportar cualquier anomalía al área de Seguridad o al líder técnico.

---

10. Contacto de soporte

📧 devops@techretail.com
☎️ Ext. 1700 – Área de Desarrollo y DevOps
🕒 Horario: Lunes a viernes, 9:00 am – 6:00 pm